

Analysing the correctness of Decomposition Algorithm

Problem:

We are proving that pyMUEdit fastICA decomposition is correct

Methods

The main method to prove correctness is to benchmark our decomposition output against an existing MATLAB decomposition. Our pyMUEdit decomposition algorithm is based upon MUEdit.

<https://github.com/simonavrillon/MUEdit/tree/main/lib>

The goal is that if we benchmark pyMUEdit (python) against MUEdit (MATLAB), we can guarantee that pyMUEdit is correct. There are a couple methods that have been tried to test and verify the correctness.

If you follow the 'Running MUEdit Comparison Tests' instruction in the README, you will encounter unit tests that test individual functions. Importantly, not all of them pass, Convolutional Sphering has been commented out. The previous capstone couldn't get that test to work.

Additionally, when following the comparison test instructions, you will have a matlab_output and python_output. Inside that folder you will have the decomposed file for matlab and python. There is also a debug_output folder that will dump all the intermediate results for each step of the decomposition.

That was the previous capstone's work. My work starts here.

Speaking to the clients, they wished to have a visual overlay of the pyMUEdit and MATLAB decomposition. This is mainly to verify if the graphs match up. It is a quick and easy inspection.

How to use the overlay feature:

You will need to produce the matlab_output and python_output folders. Inside you have the decomposed file.

1. Go to muediting tab
2. Upload the python_output decomposed file
3. Then in Option -> Visualization -> Plot Option -> Overlay Benchmark Decomposition
4. Upload the matlab_output decomposed file

5. On the side bar select only MU at a time to see the overlay between the graphs

Go to the appendix to see screenshots of this to confirm. Overall, clients commented that there are differences. They noted that the spike train, pyMUEdit, seems to have a lower amplitude. The next task is to identify where in pyMUEdit decomposition does it start to diverge from MATLAB.

Using the tests/compareDebugOutput.py script

Once you've decomposed the dataset using pyMUEdit and MATLAB as outlined in the README. You will have two folders, matlab_output and python_output. Each of these folders has intermediate output. This intermediate output is the results after each step/ function in the decomposition process. So if you compare the debug output of pyMUEdit and MATLAB, you can find where they start to diverge.

Using compareDebugOutput

Run python comparDebugOutput.py

Assumes that you have a python_output and matlab_output folder already.

This script is meant to compare each intermediate output between pyMUedit and MATLAB. We are analysing numerical differences based on correlation coefficient, norm ratio and angle deg.

matlab_output															
id	name	type	value	id	name	type	value	id	name	type	value	id	name	type	value
1	1.0	float	1.0	1	1.0	float	1.0	1	1.0	float	1.0	1	1.0	float	1.0
2	2.0	float	2.0	2	2.0	float	2.0	2	2.0	float	2.0	2	2.0	float	2.0
3	3.0	float	3.0	3	3.0	float	3.0	3	3.0	float	3.0	3	3.0	float	3.0
4	4.0	float	4.0	4	4.0	float	4.0	4	4.0	float	4.0	4	4.0	float	4.0
5	5.0	float	5.0	5	5.0	float	5.0	5	5.0	float	5.0	5	5.0	float	5.0
6	6.0	float	6.0	6	6.0	float	6.0	6	6.0	float	6.0	6	6.0	float	6.0
7	7.0	float	7.0	7	7.0	float	7.0	7	7.0	float	7.0	7	7.0	float	7.0
8	8.0	float	8.0	8	8.0	float	8.0	8	8.0	float	8.0	8	8.0	float	8.0
9	9.0	float	9.0	9	9.0	float	9.0	9	9.0	float	9.0	9	9.0	float	9.0
10	10.0	float	10.0	10	10.0	float	10.0	10	10.0	float	10.0	10	10.0	float	10.0
11	11.0	float	11.0	11	11.0	float	11.0	11	11.0	float	11.0	11	11.0	float	11.0
12	12.0	float	12.0	12	12.0	float	12.0	12	12.0	float	12.0	12	12.0	float	12.0
13	13.0	float	13.0	13	13.0	float	13.0	13	13.0	float	13.0	13	13.0	float	13.0
14	14.0	float	14.0	14	14.0	float	14.0	14	14.0	float	14.0	14	14.0	float	14.0
15	15.0	float	15.0	15	15.0	float	15.0	15	15.0	float	15.0	15	15.0	float	15.0
16	16.0	float	16.0	16	16.0	float	16.0	16	16.0	float	16.0	16	16.0	float	16.0
17	17.0	float	17.0	17	17.0	float	17.0	17	17.0	float	17.0	17	17.0	float	17.0
18	18.0	float	18.0	18	18.0	float	18.0	18	18.0	float	18.0	18	18.0	float	18.0
19	19.0	float	19.0	19	19.0	float	19.0	19	19.0	float	19.0	19	19.0	float	19.0
20	20.0	float	20.0	20	20.0	float	20.0	20	20.0	float	20.0	20	20.0	float	20.0
21	21.0	float	21.0	21	21.0	float	21.0	21	21.0	float	21.0	21	21.0	float	21.0
22	22.0	float	22.0	22	22.0	float	22.0	22	22.0	float	22.0	22	22.0	float	22.0
23	23.0	float	23.0	23	23.0	float	23.0	23	23.0	float	23.0	23	23.0	float	23.0
24	24.0	float	24.0	24	24.0	float	24.0	24	24.0	float	24.0	24	24.0	float	24.0
25	25.0	float	25.0	25	25.0	float	25.0	25	25.0	float	25.0	25	25.0	float	25.0
26	26.0	float	26.0	26	26.0	float	26.0	26	26.0	float	26.0	26	26.0	float	26.0
27	27.0	float	27.0	27	27.0	float	27.0	27	27.0	float	27.0	27	27.0	float	27.0
28	28.0	float	28.0	28	28.0	float	28.0	28	28.0	float	28.0	28	28.0	float	28.0
29	29.0	float	29.0	29	29.0	float	29.0	29	29.0	float	29.0	29	29.0	float	29.0
30	30.0	float	30.0	30	30.0	float	30.0	30	30.0	float	30.0	30	30.0	float	30.0
31	31.0	float	31.0	31	31.0	float	31.0	31	31.0	float	31.0	31	31.0	float	31.0
32	32.0	float	32.0	32	32.0	float	32.0	32	32.0	float	32.0	32	32.0	float	32.0
33	33.0	float	33.0	33	33.0	float	33.0	33	33.0	float	33.0	33	33.0	float	33.0
34	34.0	float	34.0	34	34.0	float	34.0	34	34.0	float	34.0	34	34.0	float	34.0
35	35.0	float	35.0	35	35.0	float	35.0	35	35.0	float	35.0	35	35.0	float	35.0
36	36.0	float	36.0	36	36.0	float	36.0	36	36.0	float	36.0	36	36.0	float	36.0
37	37.0	float	37.0	37	37.0	float	37.0	37	37.0	float	37.0	37	37.0	float	37.0
38	38.0	float	38.0	38	38.0	float	38.0	38	38.0	float	38.0	38	38.0	float	38.0
39	39.0	float	39.0	39	39.0	float	39.0	39	39.0	float	39.0	39	39.0	float	39.0
40	40.0	float	40.0	40	40.0	float	40.0	40	40.0	float	40.0	40	40.0	float	40.0
41	41.0	float	41.0	41	41.0	float	41.0	41	41.0	float	41.0	41	41.0	float	41.0
42	42.0	float	42.0	42	42.0	float	42.0	42	42.0	float	42.0	42	42.0	float	42.0
43	43.0	float	43.0	43	43.0	float	43.0	43	43.0	float	43.0	43	43.0	float	43.0
44	44.0	float	44.0	44	44.0	float	44.0	44	44.0	float	44.0	44	44.0	float	44.0
45	45.0	float	45.0	45	45.0	float	45.0	45	45.0	float	45.0	45	45.0	float	45.0
46	46.0	float	46.0	46	46.0	float	46.0	46	46.0	float	46.0	46	46.0	float	46.0
47	47.0	float	47.0	47	47.0	float	47.0	47	47.0	float	47.0	47	47.0	float	47.0
48	48.0	float	48.0	48	48.0	float	48.0	48	48.0	float	48.0	48	48.0	float	48.0
49	49.0	float	49.0	49	49.0	float	49.0	49	49.0	float	49.0	49	49.0	float	49.0
50	50.0	float	50.0	50	50.0	float	50.0	50	50.0	float	50.0	50	50.0	float	50.0
51	51.0	float	51.0	51	51.0	float	51.0	51	51.0	float	51.0	51	51.0	float	51.0
52	52.0	float	52.0	52	52.0	float	52.0	52	52.0	float	52.0	52	52.0	float	52.0
53	53.0	float	53.0	53	53.0	float	53.0	53	53.0	float	53.0	53	53.0	float	53.0
54	54.0	float	54.0	54	54.0	float	54.0	54	54.0	float	54.0	54	54.0	float	54.0
55	55.0	float	55.0	55	55.0	float	55.0	55	55.0	float	55.0	55	55.0	float	55.0
56	56.0	float	56.0	56	56.0	float	56.0	56	56.0	float	56.0	56	56.0	float	56.0
57	57.0	float	57.0	57	57.0	float	57.0	57	57.0	float	57.0	57	57.0	float	57.0
58	58.0	float	58.0	58	58.0	float	58.0	58	58.0	float	58.0	58	58.0	float	58.0
59	59.0	float	59.0	59	59.0	float	59.0	59	59.0	float	59.0	59	59.0	float	59.0
60	60.0	float	60.0	60	60.0	float	60.0	60	60.0	float	60.0	60	60.0	float	60.0
61	61.0	float	61.0	61	61.0	float	61.0	61	61.0	float	61.0	61	61.0	float	61.0
62	62.0	float	62.0	62	62.0	float	62.0	62	62.0	float	62.0	62	62.0	float	62.0
63	63.0	float	63.0	63	63.0	float	63.0	63	63.0	float	63.0	63	63.0	float	63.0
64	64.0	float	64.0	64	64.0	float	64.0	64	64.0	float	64.0	64	64.0	float	64.0
65	65.0	float	65.0	65	65.0	float	65.0	65	65.0	float	65.0	65	65.0	float	65.0
66	66.0	float	66.0	66	66.0	float	66.0	66	66.0	float	66.0	66	66.0	float	66.0
67	67.0	float	67.0	67	67.0	float	67.0	67	67.0	float	67.0	67	67.0	float	67.0
68	68.0	float	68.0	68	68.0	float	68.0	68	68.0	float	68.0	68	68.0	float	68.0
69	69.0	float	69.0	69	69.0	float	69.0	69	69.0	float	69.0	69	69.0	float	69.0
70	70.0	float	70.0	70	70.0	float	70.0	70	70.0	float	70.0	70	70.0	float	70.0
71	71.0	float	71.0	71	71.0	float	71.0	71	71.0	float	71.0	71	71.0	float	71.0
72	72.0	float	72.0	72	72.0	float	72.0	72	72.0	float	72.0	72	72.0	float	72.0
73	73.0	float	73.0	73	73.0	float	73.0	73	73.0	float	73.0	73	73.0	float	73.0
74	74.0	float	74.0	74	74.0	float	74.0	74	74.0	float	74.0	74	74.0	float	74.0
75	75.0	float	75.0	75	75.0	float	75.0	75	75.0	float	75.0	75	75.0	float	75.0
76	76.0	float	76.0	76	76.0	float	76.0	76	76.0	float	76.0	76	76.0	float	76.0
77	77.0	float	77.0	77	77.0	float	77.0	77	77.0	float	77.0	77	77.0	float	77.0
78	78.0	float	78.0	78	78.0	float	78.0	78	78.0	float	78.0	78	78.0	float	78.0
79	79.0	float	79.0	79	79.0	float	79.0	79	79.0	float	79.0	79	79.0	float	79.0
80	80.0	float	80.0	80	80.0	float	80.0	80	80.0	float	80.0	80	80.0	float	80.0
81	81.0	float	81.0	81	81.0	float	81.0	81	81.0	float	81.0	81	81.0	float	81.0
82	82.0	float	82.0	82	82.0	float	82.0	82	82.0	float	82.0	82	82.0	float	82.0
83	83.0	float	83.0	83	83.0	float	83.0	83	83.0	float	83.0	83	83.0	float	83.0
84	84.0	float	84.0	84	84.0	float	84.0	84	84.0	float	84.0	84	84.0	float	84.0
85	85.0	float	85.0	85	85.0	float	85.0	85	85.0	float	85.0	85	85.0	float	85.0
86	86.0	float	86.0	86	86.0	float	86.0	86	86.0	float	86.0	86	86.0	float	86.0
87	87.0	float	87.0	87	87.0	float	87.0	87	87.0	float	87.0	87	87.0	float	87.0
88	88.0	float	88.0	88	88.0	float	88.0	88	88.0	float	88.0	88	88.0	float	88.0
89	89.0	float	89.0	89	89.0	float	89.0	89	89.0	float	89.0	89	89.0	float	89.0
90	90.0	float	90.0	90	90.0	float	90.0	90	90.0	float	90.0	90	90.0	float	90.0
91	91.0	float	91.0	91	91.0	float	91.0	91	91.0	float	91.0	91	91.0	float	91.0
92	92.0	float	92.0	92	92.0	float	92.0	92	92.0	float	92.0	92	92.0	float	92.0

These are the values you are looking for

- Correlation → closer the values are to 1, the shape is similar
- Norm Ratio → closer the values are to 1, the magnitude is similar
- Angle Deg → closer the values are to 0, the vector are in the same direction

Based on these results, I looked into electrode 1 iteration 9 subiteration 4 onwards. It seemed diverging here.

1	9	2	init_its_i			0.0	0.9999721843620480	1292438450.0	fast_ICA_and_CKC_electrode_1_interval_0_iteration_9_sub_iteration_2.mat
1	9	2	w_sep_vect_initial	0.9999800177474760	0.36312596745742200	0.9998778678302330	705.516423200018		fast_ICA_and_CKC_electrode_1_interval_0_iteration_9_sub_iteration_2.mat
1	9	3	w_sep_vect_orthonormalized	0.9999456775386290	0.5935024704921830	1.0	0.9999463504704090		fast_ICA_and_CKC_electrode_1_interval_0_iteration_9_sub_iteration_3.mat
1	9	4	w_sep_vect_after_fpa	0.02725836523038740	88.11534183830430	1.0	0.03288755882467250		fast_ICA_and_CKC_electrode_1_interval_0_iteration_9_sub_iteration_4.mat
1	9	5	fICA_source	0.03632824830725660	87.16640333752210	1.213593533483750	4.644588350278100		fast_ICA_and_CKC_electrode_1_interval_0_iteration_9_sub_iteration_5.mat
1	9	5	spikes	0.9990864202660670	1.2725696982998500	1.446676169376770	67620575645.0		fast_ICA_and_CKC_electrode_1_interval_0_iteration_9_sub_iteration_5.mat
1	9	6	CoV_initial			0.0	0.3377737430237440	0.0866695106245781	fast_ICA_and_CKC_electrode_1_interval_0_iteration_9_sub_iteration_6.mat
1	9	6	ISI	0.005632039584212520	28.531098410869300	1.3054684034251600	1.1664118766784700		fast_ICA_and_CKC_electrode_1_interval_0_iteration_9_sub_iteration_6.mat
1	9	7	w_n_p1	0.009992091206089990	89.2023216985643	0.853487062789651	32836.48140368860		fast_ICA_and_CKC_electrode_1_interval_0_iteration_9_sub_iteration_7.mat
1	9	8	CoV_after_min_cov_isi			0.0	0.706278304072986	0.03350543212402030	fast_ICA_and_CKC_electrode_1_interval_0_iteration_9_sub_iteration_8.mat
1	9	8	MU_filters_i	0.027410503801101100	88.24608322131560	1.026570804317290	60774.70612205740		fast_ICA_and_CKC_electrode_1_interval_0_iteration_9_sub_iteration_8.mat
1	9	8	spikes_after_min_cov_isi	0.9998097100280310	0.5749404193724010	0.998920278374066	96186117623.0		fast_ICA_and_CKC_electrode_1_interval_0_iteration_9_sub_iteration_8.mat
1	9	9	B_sep_mat_i	0.02725836523038740	88.11534183830430	1.0	0.03288755882467250		fast_ICA_and_CKC_electrode_1_interval_0_iteration_9_sub_iteration_9.mat
1	9	10	SIL_value			0.0	0.993300104865755	0.9053079097755380	fast_ICA_and_CKC_electrode_1_interval_0_iteration_9_sub_iteration_10.mat
1	9	10	fICA_source_after_silhouette	0.036949077668639600	87.34802966051680	0.9125723558915000	6.579893774468870		fast_ICA_and_CKC_electrode_1_interval_0_iteration_9_sub_iteration_10.mat
1	9	10	spikes_after_silhouette	0.9998843063478670	0.4498996614205740	0.9984140132987400	96288364256.0		fast_ICA_and_CKC_electrode_1_interval_0_iteration_9_sub_iteration_10.mat

Now, I was looking into whether the fixed point algorithm was where it started diverging between pyMUEdit and MATLAB. I did this by adding debug statements to both the pyMUEdit version of the fixed point algorithm and MATLAB version. Then I ran a fixed point algorithm with debug as an isolated function using the previous intermediate output.

After testing the fixed point algorithm it showed that pyMUEdit and MATLAB implementation were the same. I double checked if the numba package that sped up computation affected the results, it didn't affect the results.

This is where I ran out of time, if I had more time. I would have looked into the convolutive sphering. In hindsight, I should've started there because

1. The Unit testing didn't test convolutive sphering
2. The Unit testing saw that fixed point algorithm was correct
3. Convolutive sphering is a preprocessing before the actual Fast ICA algorithms. Perhaps, that's where they could've propagated differences

I will never know. I was looking into whiten_emg but ran out of time.

If you want to isolate functions between pyMUEdit and matlab like how I've done, MATLAB debug outputs do not output convolutive sphering outputs, you will have to add them yourself. Also, pyMUEdit whiten_emg combines MATLAB functions of pcaseig.mat and whitessig.m.

There's a chance I made an error in either testing compareDebugOutput or comparing fixed point algorithm. Because I couldn't account for these results. That's a sobering thought.

What does this mean for you?

If you've been tasked with proving the correctness of pyMUEdit, it's up to you how you do it. I can't force you to pick up where I left off. My method could've been wrong, I didn't have enough time to verify.

There's a couple ways to fix it as well. If you fix the unit testing and find which function is propagating errors, that is an equally valid method in doing this.

If you run the decomposition you will encounter this error, relating to the cutMUAP function. I didn't have time to investigate

ComplexWarning: Casting complex values to real discards the imaginary part
win[: edge len] = tmp [:edge len]

Best of luck to you,
Previous Capstone Project

Appendix

This is how you overlay matlab and python output, you can see difference in the pulse train.











